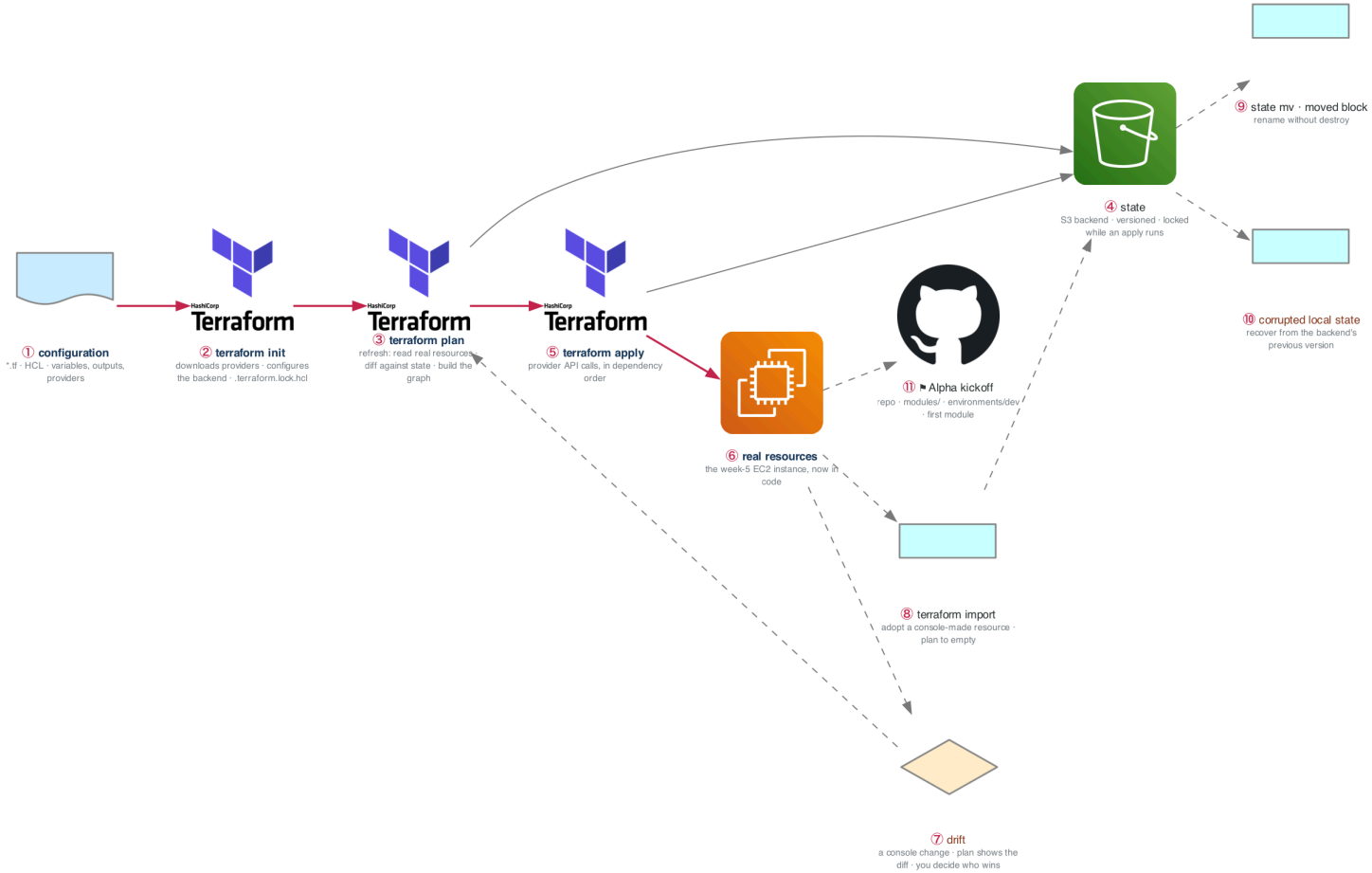
# What Terraform does when you type apply, numbered

Week 10 · Figure 10.1 · figure with badges 1–11 · 2026-10-17



# What to say, badge by badge

About three and a half minutes at a normal pace (450 words). Point at each badge as you say its line.

**Open**
“Tuesday you sit the SAA. Wednesday you start Terraform, and the reason that order works is this picture: you already know what every resource is, so all that is left is the lifecycle. Five boxes on the blue line.”

**1 configuration**
“HCL files describe the desired world: resources, variables, outputs, and which providers to use. Nothing happens by writing them. They are a declaration, not a script.”

**2 terraform init**
“Downloads the providers, pins their versions in the lock file, and configures where state will live. The lock file is committed; it is what makes two machines produce the same plan.”

**3 terraform plan**
“Three steps in one command. Refresh reads the real resources through the provider. Then Terraform diffs the configuration against the state. Then it builds a dependency graph and prints what it would do. Read the plan every time. Read the graph once, with `terraform graph`, so you believe the order.”

**4 state**
“The map from configuration addresses to real resource IDs. It lives in an S3 bucket, versioned, and a lock object stops two applies colliding. Losing state does not destroy anything, but it loses the map, and rebuilding the map is the import work in badge 8.”

**5 terraform apply**
“The provider calls the API, in dependency order, and writes each result into state as it goes. An apply that fails halfway leaves a partial state that is still correct about what exists.”

**6 real resources**
“The week-5 EC2 instance, now reproducible. Destroy it and apply it until the loop is muscle memory. That loop is what makes the clean-account test in week 13 possible.”

**7 drift**
“Change something in the console. The next plan shows the difference. Terraform does not decide who wins; you do. Drift you did not expect is an incident; drift you expected is a plan you have not applied yet.”

**8 terraform import**
“Adopt a resource someone created by hand: write the configuration, import the ID into state, and plan. The goal is a plan with no changes. That is the exit criterion, and it is how brownfield accounts get into code.”

**9 state mv and moved blocks**
“Renaming a resource in configuration looks like destroy-and-create to Terraform. A moved block, or state mv, tells it the resource only changed names. Week 11 does this across module boundaries.”

**10 corrupted local state**
“Corrupt the local copy on purpose, then recover it from the backend's previous version. Versioning on the state bucket is the recovery path, and you should have used it once before you need it.”

**11 Alpha kickoff**
“Saturday: the repository, a modules directory, a dev environment, and the first module. Nothing fancy; the structure is what matters, because weeks 11 to 13 fill it in.”

**Close**
“Declare, init, plan, apply, record. Everything else, drift and import and moved, is the state map disagreeing with the world and you deciding which one is right.”